

Implementation of AES-128 Encryption and Decryption from Scratch

immediate

June 9, 2026

Ifteir Hossain

*Computer Science and
Engineering
United International
University
Dhaka, Bangladesh*
ihossain223445@bscse.uiu.ac.bd

**Md Mahabubur
Rahman**

*Computer Science and
Engineering
United International
University
Dhaka, Bangladesh*
mrahman223589@bscse.uiu.ac.bd

Asif Mia

*Computer Science and
Engineering
United International
University
Dhaka, Bangladesh*
amia221044@bscse.uiu.ac.bd

Contents

1	Introduction	3
2	Preliminaries and Mathematical Background	3
2.1	The State Matrix	3
2.2	Galois Field $GF(2^8)$	4
2.3	Key Expansion	4
2.4	Padding	4
2.5	Python Implementation	4
2.6	Importance of the Project	5
2.6.1	AES is widely used in:	5
3	Proposed Algorithm and Methodology	5
3.1	System Flow Architecture	5
3.2	Core Transformations	5
3.3	Key Expansion (The Key Schedule)	6
3.4	Encryption Methodology (Step-by-Step)	6
3.5	Decryption Methodology (Inverse Process)	7
3.6	Data Padding Standard	7
3.7	Algorithm and Flowchart	7
3.8	Flowchart of the Process	7
3.8.1	AES-128 Encryption Algorithm	8
3.8.2	AES-128 Encryption Flowchart	9
3.9	Conceptual Flow Chart for AES (128, 192, 256)	10
3.9.1	Comparison of AES Variants	10
4	Step-by-Step Execution: Case Study ("Hello!")	11
4.1	Initial Setup and Pre-processing	11
4.2	Encryption Process (11 Steps/Rounds)	11
4.3	Decryption Process (Reversing the Steps)	12
4.4	Recovery of Original Message	12
5	Implementation, Execution Guide, and Results	12
5.1	Programming Environment	12
5.2	How to Run the Project	12
5.3	Required Screenshots to Add	13
6	Conclusion	14

1 Introduction

In today's digital world, data security has become one of the most important concerns in computer systems and communication networks. Every day, millions of users exchange sensitive information such as passwords, banking information, personal messages, and confidential documents through the internet. Without proper protection, attackers can intercept, modify, or steal this information. Therefore, secure communication and data protection are essential in modern computing systems.

Cryptography is one of the most effective techniques used to protect digital information. It converts readable data into an unreadable form so that unauthorized users cannot understand the information. Only authorized users who possess the correct key can recover the original data. Encryption and decryption are the core processes of cryptography that ensure confidentiality and security of data.

Among various cryptographic algorithms, Advanced Encryption Standard is one of the most secure and widely used encryption standards in the world. AES is used in online banking systems, wireless communication, secure websites, cloud storage, file encryption, military communication, and many cybersecurity applications. Due to its high security and efficiency, AES has become the global standard for symmetric encryption.

This project focuses on implementing AES-128 encryption and decryption from scratch using Python programming language. The main purpose of this project is to understand the internal working mechanism of the AES algorithm instead of relying on external cryptographic libraries. The project demonstrates how plaintext is transformed into ciphertext through multiple rounds of encryption operations and later restored back to its original form using decryption techniques.

The implementation includes important AES operations such as SubBytes, ShiftRows, MixColumns, AddRoundKey, Key Expansion, and PKCS7 Padding. Both encryption and decryption processes are implemented to provide a complete understanding of symmetric key cryptography.

This project is highly important for learning computer security and cryptography because it provides practical knowledge about how modern encryption systems work internally. It helps students develop a strong foundation in cybersecurity concepts and secure communication systems.

2 Preliminaries and Mathematical Background

To implement AES-128, several core mathematical concepts are utilized.

2.1 The State Matrix

AES operates on a 4×4 matrix of bytes known as the *State*. All transformations are applied to this matrix.

$$\text{State} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} \quad (1)$$

2.2 Galois Field $GF(2^8)$

AES uses finite field arithmetic in $GF(2^8)$. Addition is performed using the bitwise XOR operation. Multiplication is performed modulo the irreducible polynomial:

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (2)$$

2.3 Key Expansion

A 128-bit master key is expanded into 11 round keys using the AES Key Schedule, which involves **RotWord**, **SubWord**, and **Rcon** transformations.

2.4 Padding

AES requires data blocks of fixed length (16 bytes). If the plaintext length is smaller than 16 bytes, padding is added. This project uses PKCS7 padding.

Example:

Plaintext: "ifteir"

Length = 6 bytes

Padding Required = 10 bytes

Padding ensures the plaintext fits the AES block size requirement.

2.5 Python Implementation

This project is implemented using Python programming language. The implementation manually performs AES operations instead of using built-in cryptographic libraries.

The project includes:

- AES Encryption Module
- AES Decryption Module
- S-box implementation
- Key scheduling
- Padding mechanism
- Galois Field operation

The implementation helps understand the internal structure and mathematical operations of AES.

2.6 Importance of the Project

This project is important because:

- Provides practical understanding of encryption
- Demonstrates real-world cybersecurity concepts
- Helps to understand the internal workings of AES
- Strengthens knowledge of computer security
- Builds foundation for advanced cryptography research

2.6.1 AES is widely used in:

- Secure web communication (HTTPS)
- Wireless security
- Banking systems
- Cloud storage
- Secure file transfer

3 Proposed Algorithm and Methodology

The methodology of this project is based on the iterative process of **Rijndael Cipher** for a 128-bit block. The system is designed as a modular Python implementation that performs encryption and decryption by manipulating a 4×4 state matrix through various mathematical layers.

3.1 System Flow Architecture

The overall process is divided into three main phases:

- **Pre-processing Phase:** Plaintext padding and state matrix initialization.
- **Key Scheduling Phase:** Expansion of the 128-bit master key into 11 round keys.
- **Transformation Phase:** Sequential execution of substitution, transposition, and mixing layers over 10 rounds.

3.2 Core Transformations

- **SubBytes:** Each byte is replaced using a non-linear S-box.
- **ShiftRows:** Circularly shifts the rows of the state.
- **MixColumns:** Linearly mixes each column of the state.
- **AddRoundKey:** XORs the state with the current round key.

3.3 Key Expansion (The Key Schedule)

AES-128 requires a unique key for each of its 11 rounds (Round 0 to Round 10). The original 16-byte key is expanded using the following operations:

- **RotWord** : Performs a cyclic permutation on a 4-byte word $[a_0, a_1, a_2, a_3]$ resulting in $[a_1, a_2, a_3, a_0]$
- **SubWord**: Substitutes each byte of the word using the AES S-Box.
- **Rcon (Round Constant)**: Each expanded word is XORed with a constant to break the symmetry and ensure each round key is unique.

3.4 Encryption Methodology (Step-by-Step)

The encryption process follows an iterative structure where each round (except the final round) consists of four layers:

- **SubBytes (Substitution)**: Each byte of the State matrix is replaced by a corresponding byte from the S-Box (Substitution Box). This provides non-linearity to the cipher, making it resistant to linear and differential cryptanalysis.

– Math: $S_{i,j} \leftarrow \text{S-Box}[S_{i,j}]$

1

- **ShiftRows (Transposition)**: The rows of the state matrix are cyclically shifted to the left:
 1. **Row 0**: No shift.
 2. **Row 1**: Shifted 1 byte to the left.
 3. **Row 2**: Shifted 2 bytes to the left.
 4. **Row 3**: Shifted 3 bytes to the left. This provides diffusion by spreading the influence of bytes across different columns
- **MixColumns (Linear Transformation)**: Each column of the state matrix is multiplied by a fixed polynomial in the Galois Field $GF(2^8)$. This is the most computationally intensive part and ensures that each byte in the ciphertext is influenced by multiple bytes from the plaintext.
 - This step is omitted in the 10th or final round per the AES standard
- **AddRoundKey**: A bitwise XOR operation is performed between the current State matrix and the Round Key generated for that specific round. This is the only step where the Secret Key is directly mixed with the data.

3.5 Decryption Methodology (Inverse Process)

Decryption is achieved by reversing the encryption steps in the exact opposite order using **Inverse Transformations**:

- **InvShiftRows**: Shifting rows to the right.
- **InvSubBytes**: Using the Inverse S-Box table.
- **InvMixColumns**: Matrix multiplication using the inverse fixed polynomial in $GF(2^8)$.
- **AddRoundKey**: Remains the same (XOR is its own inverse).

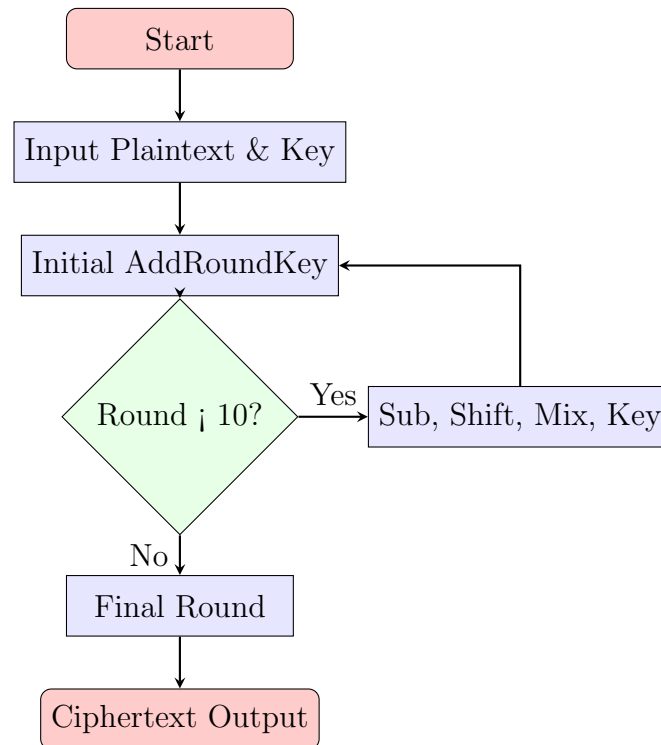
3.6 Data Padding Standard

To handle plaintext of any length, this project implements the **PKCS7 Padding Standard**.

- If the plaintext is L bytes, and L is not a multiple of 16, the algorithm calculates $Pad = 16 - (L \bmod 16)$.
- Pad number of bytes are added to the plaintext, each carrying the value of Pad .
- This ensures the input block is always 128 bits (16 bytes).

3.7 Algorithm and Flowchart

3.8 Flowchart of the Process



3.8.1 AES-128 Encryption Algorithm

AES-128 encryption works through a repeated step-by-step process called an **iterative process**. The algorithm is explained below :

1. **Input:**

- a 16-byte Plaintext (original message)
- a 16-byte Cipher Key (secret key)

2. **Key Expansion:** The original key is expanded to create **11 Round Keys**.

3. **State Initialization:** Convert the plaintext into a **4×4 matrix**, called the **State Matrix**.

4. **Initial Round**

- **AddRoundKey:** Perform an XOR operation between the State Matrix and Round Key[0].

5. **Main Rounds (Round 1 to Round 9):** In each round, perform these four steps:

- **SubBytes:** Replace each byte using an S-Box substitution table.
- **ShiftRows:** Shift the rows of the matrix left by different positions.
- **MixColumns:** Mix the columns using matrix multiplication in the Galois Field.
- **AddRoundKey:** XOR the current state with the current Round Key.

6. **Final Round (Round 10):** Perform:

- SubBytes
- ShiftRows
- AddRoundKey

The MixColumns step is skipped in the final round

7. **Output:** Convert the final State Matrix back into a one-dimensional array. This final encrypted result is called the **Ciphertext**.

3.8.2 AES-128 Encryption Flowchart

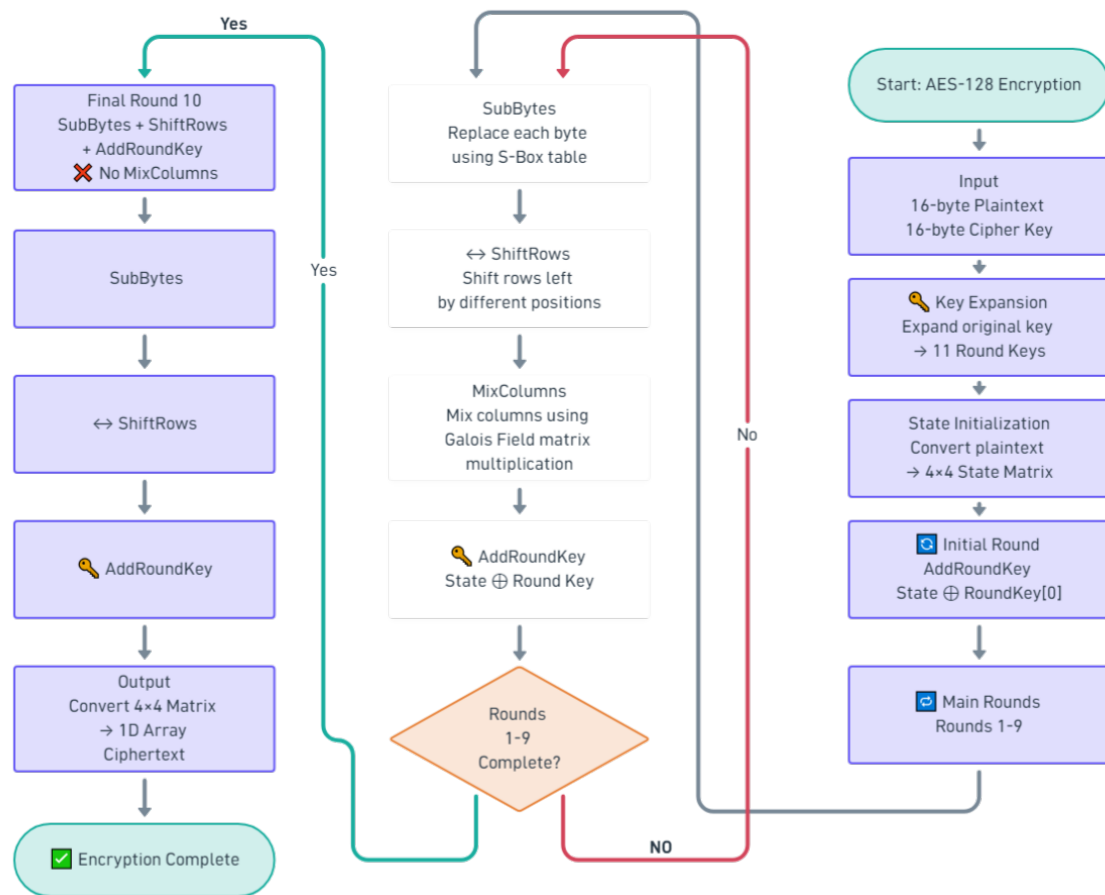


Figure 1: AES-128 Encryption

3.9 Conceptual Flow Chart for AES (128, 192, 256)

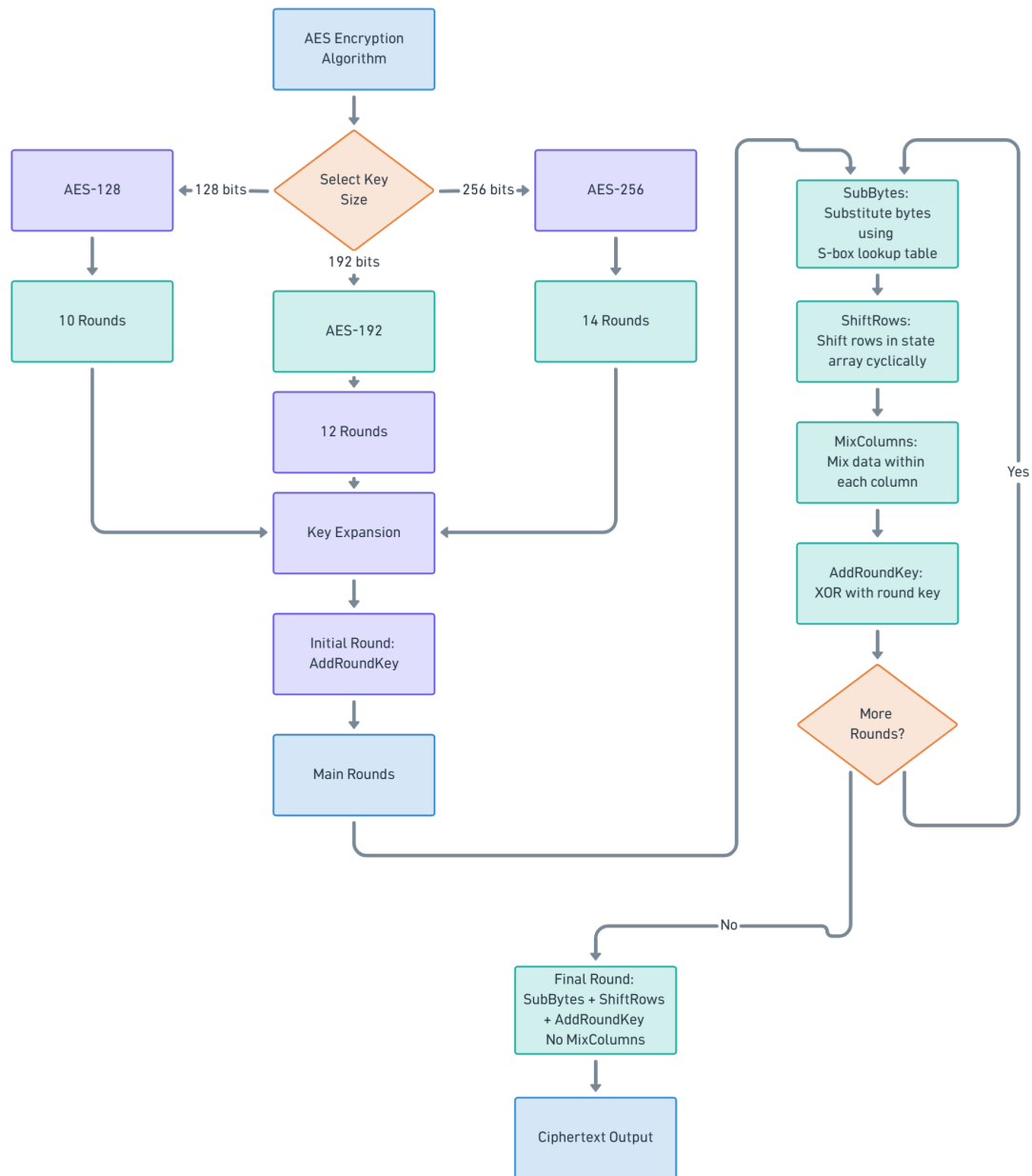


Figure 2: Conceptual Flow Chart

3.9.1 Comparison of AES Variants

Feature	AES-128	AES-128	AES-256
Key Size	128 bits (16 bytes)	192 bits (24 bytes)	256 bits (32 bytes)
Block Size	128 bits	128 bits	128 bits
Number of Rounds	10	12	14
Round Keys	11	13	15

4 Step-by-Step Execution: Case Study ("Hello!")

In this section, we present a practical, step-by-step trace of the implemented AES-128 algorithm using the exact test case from our experimental logs. This serves as a manual validation of the Python implementation's internal logic.

4.1 Initial Setup and Pre-processing

Before the iterative transformation rounds begin, the input string must be formatted to comply with the 16-byte block constraint of the AES standard

- **Original Message:** Hello! (6 bytes)
- **Cipher Key:** MySecretKey12345 (16 bytes)
- **Padding (PKCS7 Standard):** Since the block requires 16 bytes, the system automatically calculates the remainder ($16 - 6 = 10$ bytes). Ten padding bytes, each containing the hexadecimal value 0x0a (decimal 10), are appended.
- **Input Block (Padded Hex):** 48 65 6c 6c 6f 21 0a 0a 0a 0a 0a 0a 0a 0a 0a

4.2 Encryption Process (11 Steps/Rounds)

The execution flow operates over exactly 11 states (Round 0 through Round 10):

- **Step 0: Initial Round (AddRoundKey):** The padded input block matrix is bitwise XORed with the master key (Round Key 0). This introduces the secret key into the state right at the beginning.
- **Step 1 to 9: Main Rounds:** For these nine intermediary rounds, the state matrix undergoes the full sequential pipeline:
 - **SubBytes:** Byte substitution via the non-linear S-box to achieve confusion.
 - **ShiftRows:** Cyclic row transposition to spread bytes across different columns.
 - **MixColumns:** Linear column mixing via Galois Field arithmetic ($GF(2^8)$) to achieve maximum bit diffusion
 - **AddRoundKey:** XORing the state with the unique key generated for that specific round.
- **Step 10: Final Round:** Per the standard architectural specifications, the MixColumns step is skipped. The state passes only through SubBytes → ShiftRows → AddRoundKey
- **Final Output (Ciphertext):** 5cee9c0575b25893774cd5f355bc30d4

4.3 Decryption Process (Reversing the Steps)

The decryption sequence reverses the entire operations matrix using inverse transformations to retrieve the original state safely :

- **Rounds 10 to 1:** Executed using AddRoundKey $\rightarrow$ InvMixColumns (skipped in Round 10) $\rightarrow$ InvSubBytes $\rightarrow$ InvShiftRows
- **Round 0:** final execution of AddRoundKey using the original master key completes the mathematical reversal.

4.4 Recovery of Original Message

- **Decryption Output (Hex):** 48 65 6c 6c 6f 21 0a 0a 0a 0a 0a 0a 0a 0a 0a
- **Unpadding:** The parser scans the trailing block, reads the final 0x0a byte, recognizes 10 bytes of padding, and strips them off safely.
- **Final Plaintext Result:** Hello!

5 Implementation, Execution Guide, and Results

The project was implemented in Python. The program successfully encrypts a 128-bit block and decrypts it back to the original message using PKCS7 padding.

5.1 Programming Environment

The environment layout utilized for testing and running this implementation is detailed below:

- **Language/Engine:** Python
- **Libraries:** None (Built entirely from scratch using native arrays and standard mathematical operators to study the low-level logic).

5.2 How to Run the Project

To execute the cryptographic system, follow these precise terminal steps:

1. Save the target file as `aes_system.py` inside your working directory.
1. Open your operating system's Terminal or Command Prompt (CMD).
2. Navigate to the project directory and run the execution trigger:

Bash

```
python aes_system.py
```

Figure 3: Bash Code

5.3 Required Screenshots to Add

```
S_BOX = [  
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,  
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0,  
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15,  
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,  
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84,  
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,  
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8,  
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2,  
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73,  
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb,  
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79,  
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08,  
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a,  
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e,  
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf,  
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16  
]
```

Figure 4: showing custom S-Box matrices

```
def galois_mult(a, b):
    """Galois field multiplication"""
    p = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        hi_bit_set = a & 0x80
        a <<= 1
        if hi_bit_set:
            a ^= 0x1b # Reduce with AES polynomial
        b >>= 1
    return p & 0xFF
```

Figure 5: Finite Field Multiplier logic in the IDE

```
=====
Original plaintext: b'Hello!' (6 bytes)
Key:                b'MySecretKey12345'
Ciphertext:         5cee9c0575b25893774cd5f355bc30d4

[Done] exited with code=0 in 0.172 seconds
```

Figure 6: Execution trace showing the output logs of the input string "Hello!"

```
[Running] python -u "c:\Users\iftei\OneDrive\Desktop\Computer Security project\AESdecryption.py"
Ciphertext: 5cee9c0575b25893774cd5f355bc30d4
Decrypted Plaintext: b'Hello!'

[Done] exited with code=0 in 0.165 seconds
```

Figure 7: Final console output showcasing the Ciphertext and successful unpadded text reconstruction

6 Conclusion

The core structural implementation of the AES-128 cryptographic system from scratch has been executed successfully. By engineering the software without external third-party dependencies, we verified how cryptographic primitive layers cooperate to provide robust data protection.

The test case metrics confirm that implementing precise non-linear byte substitutions combined with modular matrix arithmetic yields a highly distributed diffusion profile. This project successfully achieves its learning objective, building a crucial technical bridge between structural mathematical theory and low-level software application within contemporary cybersecurity engineering.